



North South University

Department of Electrical & Computer Engineering

Assignment

Course Code: CSE 411

Course Title: Advanced Database Systems

Faculty Name: Dr. Abu Sayed Md. Latiful Hoque

Submission Date: 17-12-2025

Submitted by

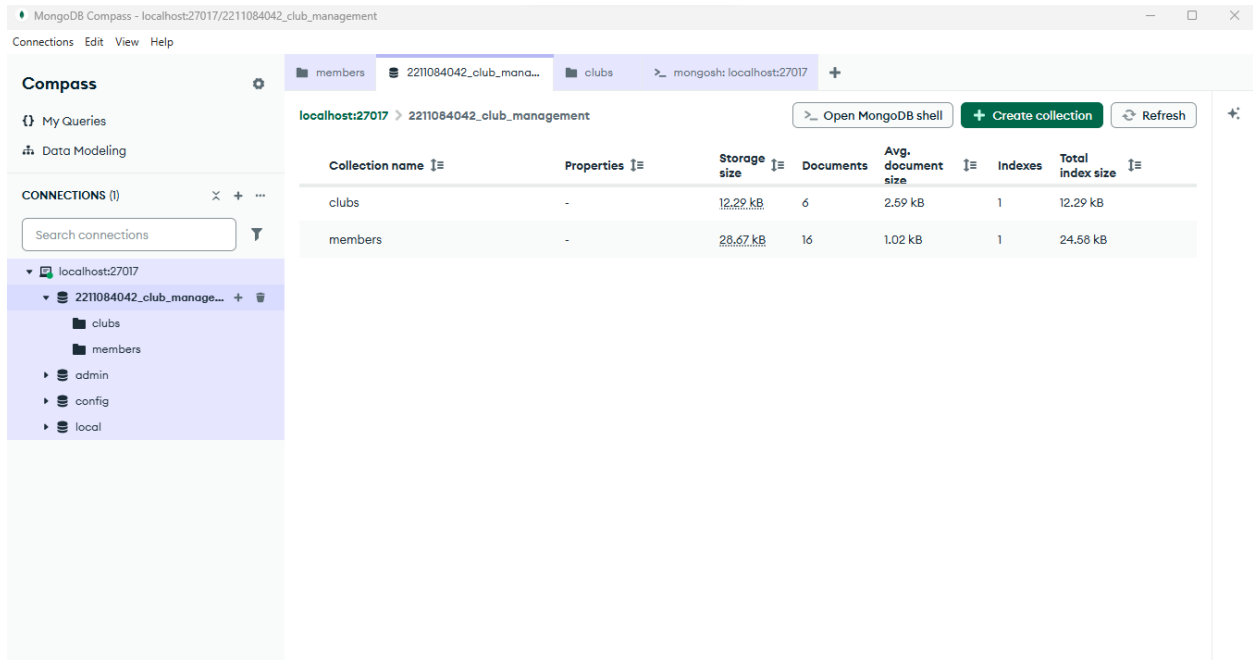
Name: Israt Zaman Srity

ID: 2211084042

Task 1:

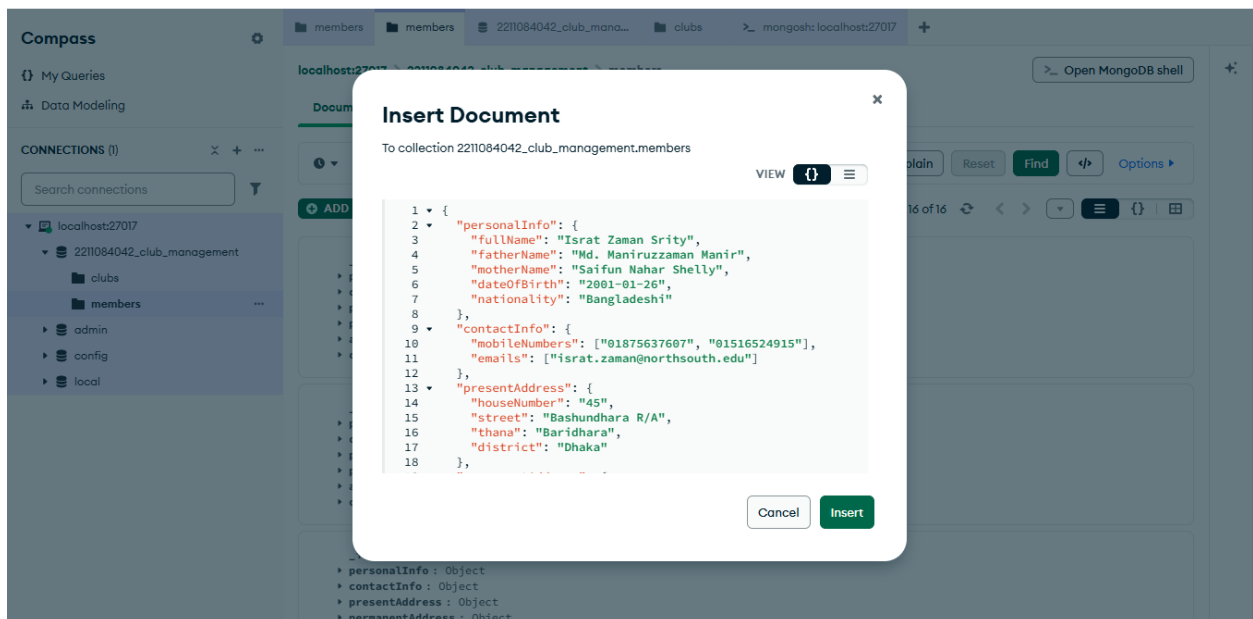
the database named club_management with your Student ID as the prefix.
(2211084042_club_management)

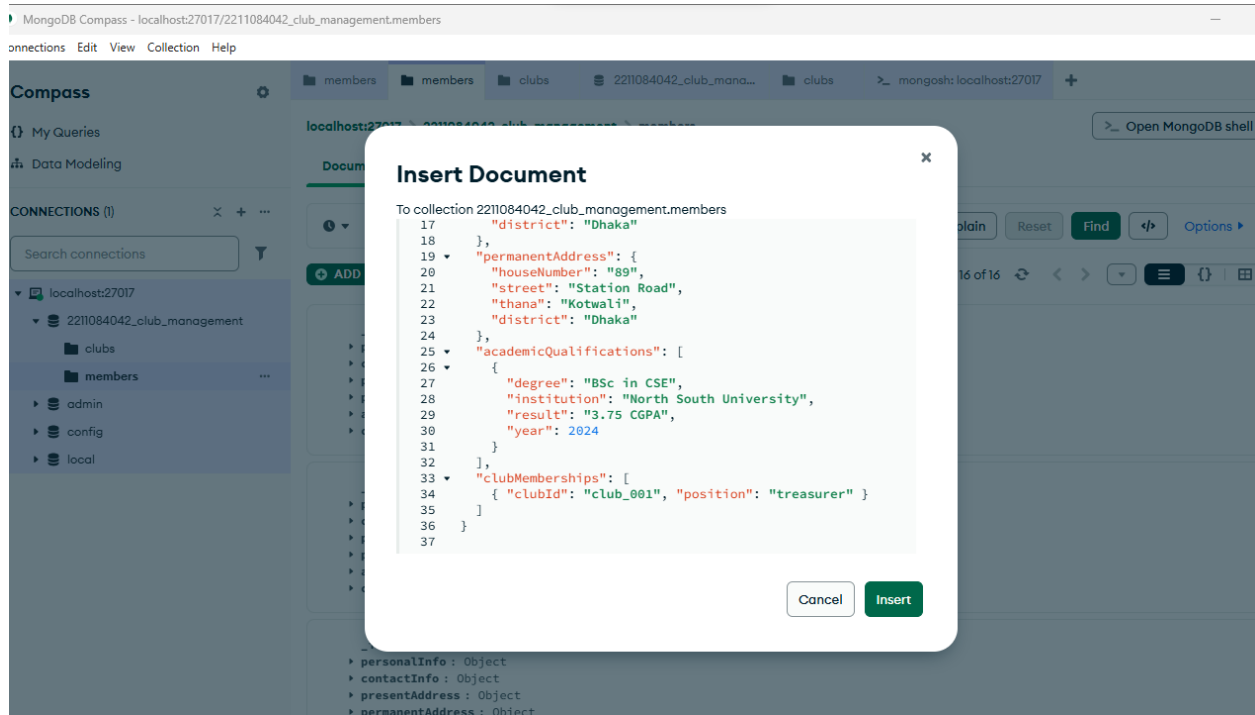
the two collections clubs and members



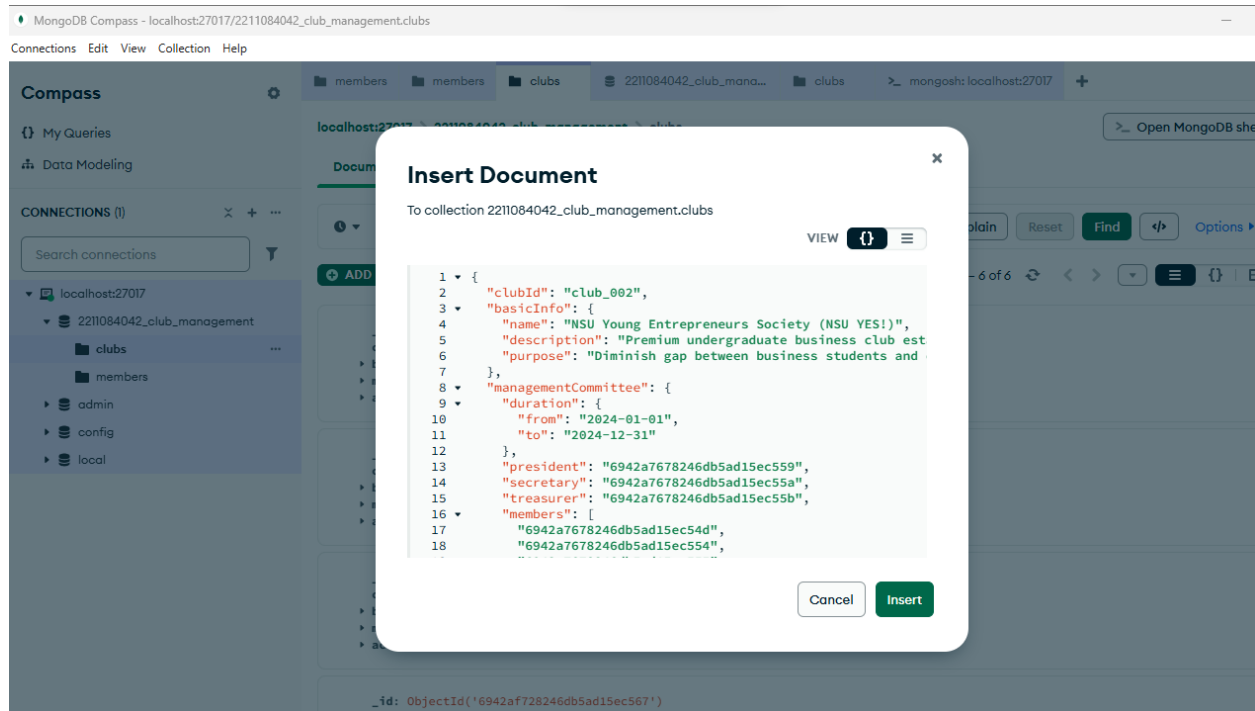
Task 2: Insert Data

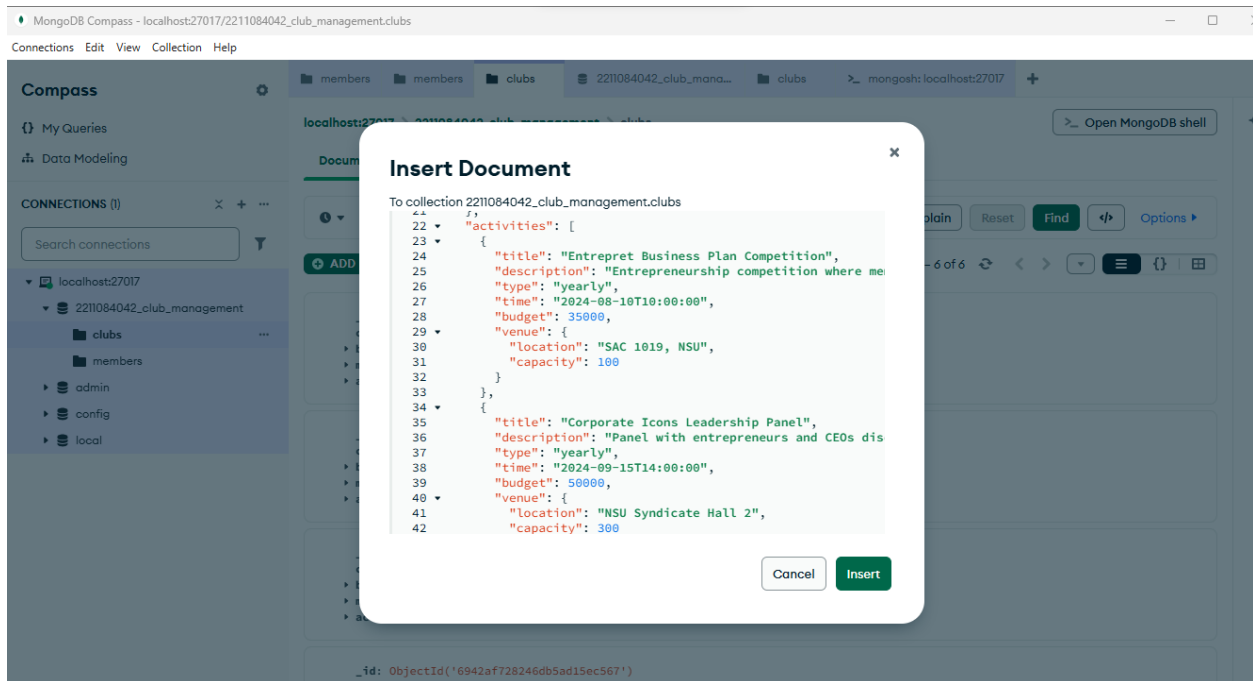
Members:





Club:



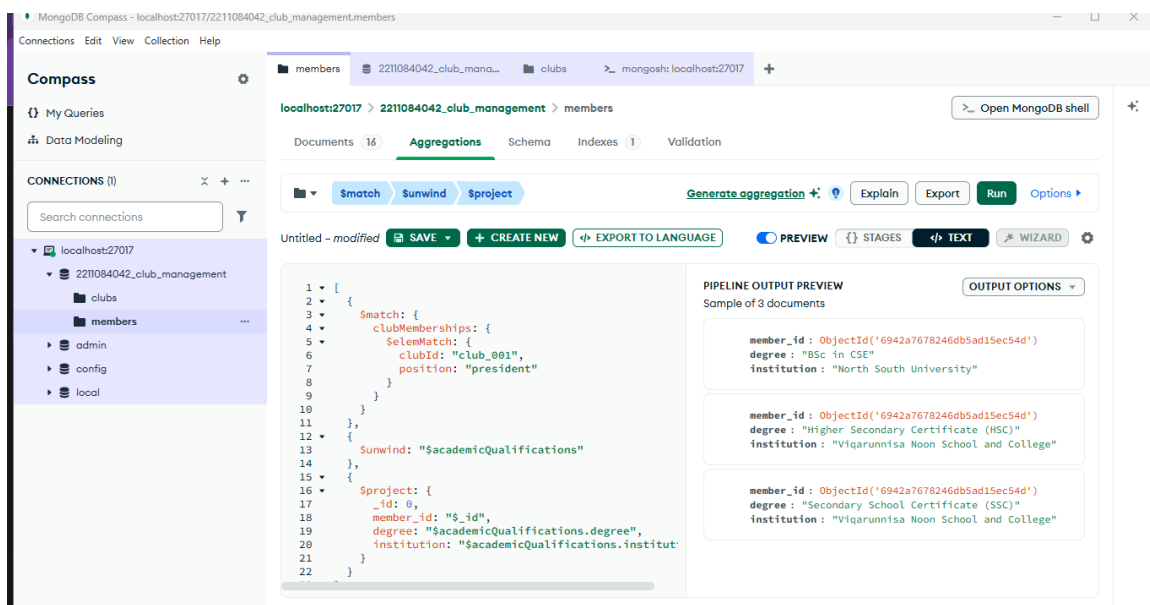


json files containing all MongoDB Documents (Memembrs and clubs)

<https://drive.google.com/drive/folders/1TqyIS0ansRzqzPaP5hgHN2mOJEbHfn4k?usp=sharing>

Output of the queries

Query1: Find member ID, Degree, institute of the president of club 01.



Query2: Find the clubid, Club name and president name of member name ...

The screenshot shows the MongoDB Compass interface with the 'clubs' collection selected. The aggregation pipeline is defined as follows:

```
13   {
14     $lookup: {
15       foreignField: "_id",
16       as: "president"
17     },
18     $unwind: "$president"
19   },
20   {
21     $match: {
22       "president.personalInfo.fullName": "Israt Zaman"
23     },
24   },
25   {
26     $project: {
27       _id: 0,
28       clubId: 1,
29       clubName: "$basicInfo.name",
30       presidentName: "$president.personalInfo.fullName"
31     }
32   }
33 }
```

The pipeline output preview shows a sample of 1 document:

```
{
  "clubId": "club_001",
  "clubName": "NSU Computer & Engineering Club (NSU CEC)",
  "presidentName": "Israt Zaman Srity"
}
```

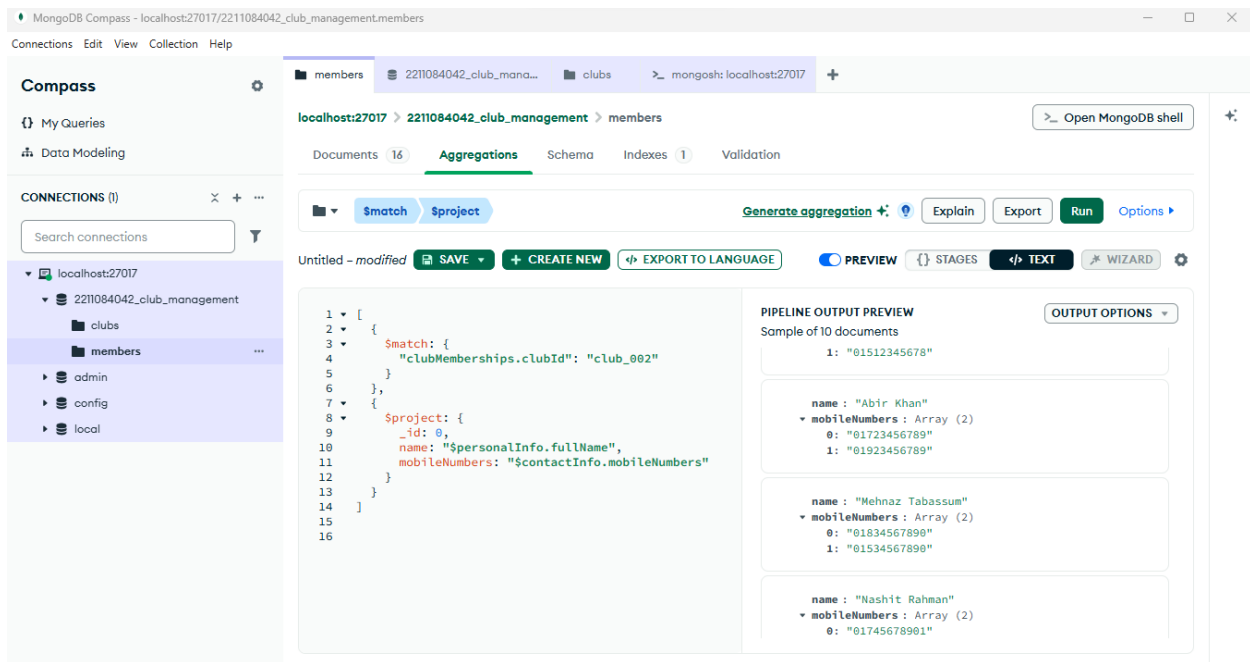
Query3: Find Name, mobile numbers of all members of club 02.

The screenshot shows the MongoDB Compass interface with the 'members' collection selected. The aggregation pipeline is defined as follows:

```
1 [
2   {
3     $match: {
4       "clubMemberships.clubId": "club_002"
5     },
6   },
7   {
8     $project: {
9       _id: 0,
10      name: "$personalInfo.fullName",
11      mobileNumbers: "$contactInfo.mobileNumbers"
12    }
13  }
14 ]
15
16 }
```

The pipeline output preview shows a sample of 10 documents:

```
{
  "name": "Israt Zaman Srity",
  "mobileNumbers": [
    "01875637687",
    "01956789012"
  ]
},
{
  "name": "Samin Yasar",
  "mobileNumbers": [
    "01756789012",
    "01956789012"
  ]
},
{
  "name": "Fabiha Bushra",
  "mobileNumbers": [
    "01778901234",
    "01978901234"
  ]
}
```



Comparison of my MongoDB implemented with the implementation using MySQL or Oracle.

If this system were implemented using a **Relational Database Management System (RDBMS)** like **MySQL or Oracle**, the design and data storage would be different from **MongoDB (NoSQL)**.

1. Structure: Collection vs Table

In MongoDB, data is stored in collections made up of documents, where each document contains fields.

In RDBMS like MySQL or Oracle, data is stored in tables consisting of rows (tuples) and columns with a fixed structure.

2. Schema Flexibility

MongoDB is schema-less, so different member documents can have different fields and easily store arrays or nested objects like club memberships.

RDBMS follows a fixed schema, where all rows must follow the same structure and complex data is split into multiple related tables.

3. Handling Relationships

MongoDB handles relationships using embedded documents or manual references, without enforced constraints. RDBMS uses foreign keys and JOIN operations, ensuring strong referential integrity between tables.

4. Data Retrieval

MongoDB retrieves data using `find()` and `aggregate()`, and does not use JOINS by default. RDBMS uses SQL SELECT statements with JOINS to combine data from multiple tables.

5. Performance & Scalability

MongoDB is horizontally scalable and performs well with large JSON-like data and big-data applications.

MySQL and Oracle are vertically scalable and are better suited for structured, transactional systems.

6. Normalization vs Denormalization

MongoDB uses denormalized data, allowing some duplication for faster read performance. RDBMS uses normalized data, reducing redundancy and maintaining strong consistency.

7. Real-World Use Case Comparison

MongoDB is better for applications with flexible data and high scalability, such as student or club management systems.

MySQL or Oracle are better for banking and financial systems that require strict ACID compliance.

Aspect	MongoDB (Used in this Project)	MySQL / Oracle
Database type	NoSQL, document-based	Relational (RDBMS)
Data structure	JSON-like documents	Tables with rows & columns
Member data storage	Complete member info in one document	Member info split into multiple tables
Club membership	Embedded inside member document	Separate membership table
Relationship handling	Embedding and referencing	Foreign keys
Join requirement	Optional (\$lookup)	Mandatory (JOIN)
Schema flexibility	Schema-less, easy to modify	Fixed schema, hard to change

